GLA UNIVERSITY / AIML DEPARTMENT

Project Management System

Four portals, four services, one shared Postgres schema: the platform that runs the final-year project lifecycle from team formation to published grade.

4 portals 313 endpoints 25+ tables 69 screens Docker / EC2

AT A GLANCE

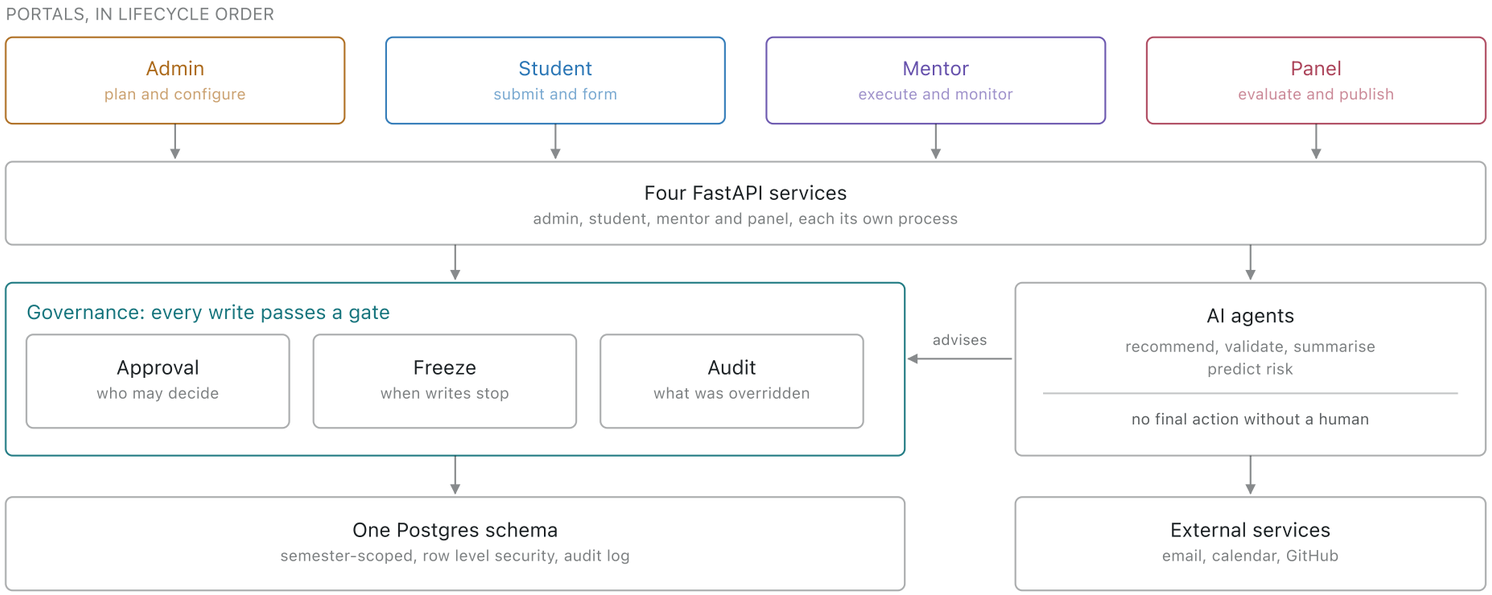
The system in numbers

313	69	25	9
HTTP ENDPOINTS	SCREENS	BASE TABLES	CONTAINERS

STUDENT API	MENTOR API	ADMIN API	PANEL API
114 endpoints / 19 screens	**98** endpoints / 14 screens	**58** endpoints / 25 screens	**43** endpoints / 11 screens

ARCHITECTURE / THE WHOLE SYSTEM

One lifecycle, four portals, three gates



Access is `role + semester + stage + ownership + policy`. Role alone never decides.

STAGE, NOT JUST ROLE

A coordinator is still a coordinator after grading closes, but the freeze gate has changed what that means. The same request is legal in one stage and rejected in the next.

AGENTS ADVISE, PEOPLE DECIDE

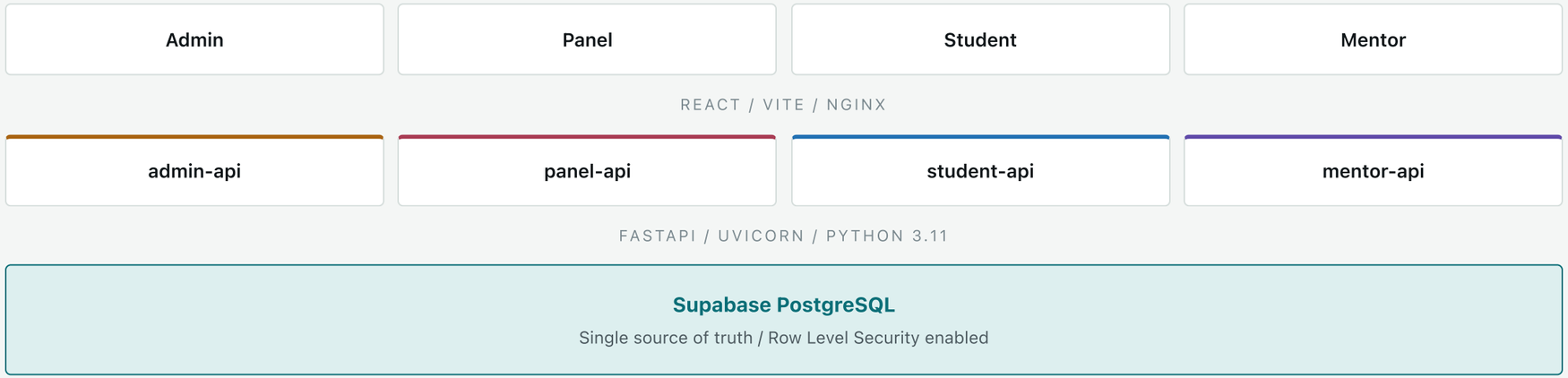
Allocation, risk and quality signals arrive as recommendations with a visible reason. Approving, assigning, finalising and publishing stay with a person.

UNLOCKING IS AN EVENT

An override is not a quiet exception. Reopening a frozen stage is a recorded action with an actor, a timestamp and a justification behind it.

ARCHITECTURE

Four stacks, one spine



Each frontend ships in its own nginx container that reverse-proxies `/api` to its service. In production: four subdomains.

DESIGN RATIONALE

Why four services, not one monolith

01 / BLAST RADIUS

Unequal trust

Students are the largest and least-trusted population; panel members need a narrow slice for two hours. A flaw in the student API cannot reach admin functionality. That code isn't loaded in that process.

02 / LOAD SHAPE

Different lifecycles

Panel is idle for weeks, then serves every evaluator at once on presentation day. Student API peaks during team formation. Separate processes scale and restart independently.

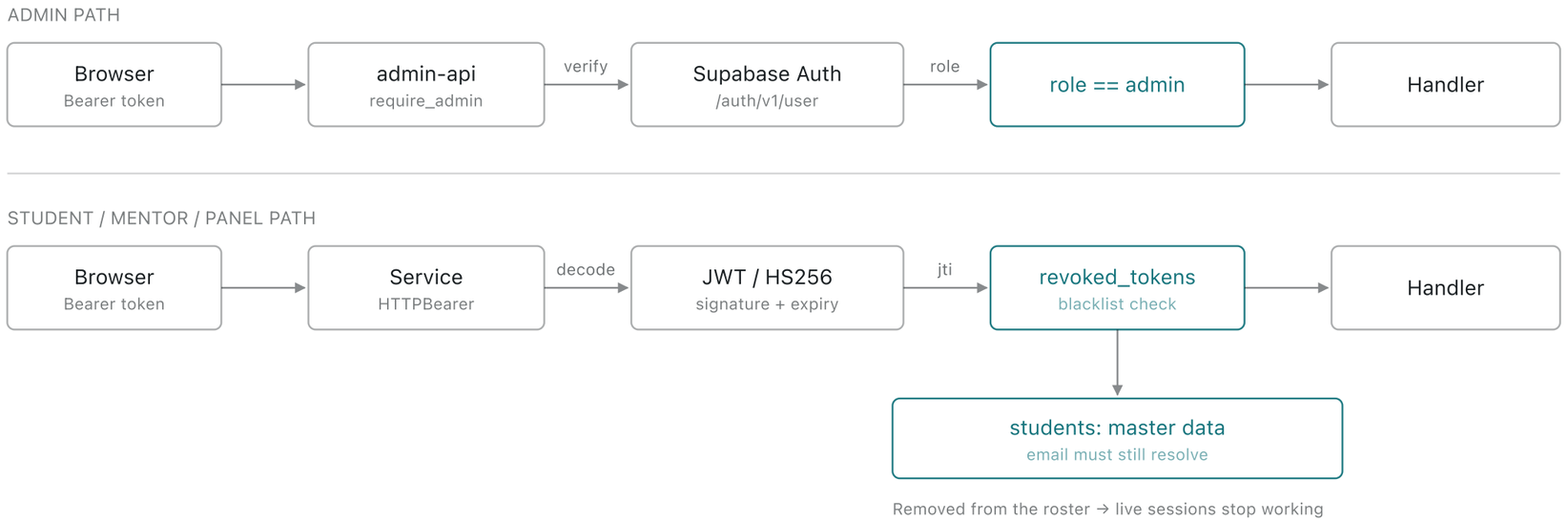
03 / IDENTITY

Two auth models

Admin authenticates through Supabase Auth. Student, mentor and panel mint their own HS256 JWTs with refresh rotation and a revocation blacklist. Cleaner apart than tangled together.

ARCHITECTURE / REQUEST LIFECYCLE

Two ways a request proves who it is



Both paths end at a handler, but they trust different authorities, and only the lower path re-checks the roster on every request.

The admin service delegates identity to **Supabase Auth**. The other three verify a token they minted themselves, then check it against the revocation blacklist, which is what makes logout actually revoke.

TECHNOLOGY / LAYER 1

Frontend

React + Vite	All five frontends. Millisecond dev starts and Rollup-optimised bundles. With five apps in one repo, slow builds compound five times over.
Tailwind CSS	A shared utility vocabulary keeps four separately-developed portals looking like one product, with no component library to maintain.
Framer Motion **Lucide / Lenis**	Animation, icons and smooth scroll, present in all five apps. This is what makes the portals read as siblings.

DIVERGENCE BY JOB

ADMIN

Data in, reports out.

`xlsx` `papaparse` `jspdf`

STUDENT

Results & repo analysis.

`recharts`

MENTOR

Progress across teams.

`chart.js` `recharts`

PANEL

Submission receipts.

`jspdf` `recharts`

TECHNOLOGY / LAYER 2

Backend

CORE

FastAPI on Uvicorn

- **Native async**: these services spend most of their time waiting on Postgres and outbound HTTP
- **Pydantic validation**: malformed input dies at the boundary, not deep in a handler
- **Automatic OpenAPI** documentation

DATA ACCESS: A DELIBERATE SPLIT

ORM where it earns its keep

- **Student & mentor:** SQLAlchemy async + asyncpg + Alembic, for complex relational models
- **Admin & panel:** Supabase client over PostgREST, for broad generic table work where an ORM adds ceremony without value

SECURITY

passlib + bcrypt: deliberately slow, salted per hash. **python-jose** for JWT.

INGESTION

pandas / openpyxl for Excel; **python-docx / pypdf** for problem statements. Parsed in memory, never written to disk.

RECORD LINKAGE

RapidFuzz + Jellyfish: names arrive from three sources with three spellings. Edit-distance plus phonetic matching.

TECHNOLOGY / LAYERS 3 & 4

Integrations & delivery

EXTERNAL SERVICES

OpenAI	Six modules: statement enrichment, domain classification, blueprint generation
Twilio	Voice and SMS reminders ahead of presentation slots
Google Calendar	Official Python client, for scheduling
Email	fastapi-mail and raw smtplib; some flows need MIME attachments

INFRASTRUCTURE

Docker Compose	Nine containers: four services, four portals, one public site
nginx	Per-frontend static serving plus `/api` reverse proxy
GitHub Actions	Platform deploys to EC2 over SSH; the public site ships on a separate Node 22 pipeline; it changes far more often and shouldn't force a stack redeploy

DATA

One schema, six clusters

IDENTITY & COHORT

`students / mentors / panels / sub_admins / semester_config`

TEAM FORMATION

`teams_validation / unassigned_students / team_join_requests`

PROJECT SELECTION

`problem_statements / proposals / project_locked_ps / project_override_logs`

MENTORSHIP

`mentor_allocations / panel_mentors`

ASSESSMENT / LARGEST

`schedule_slots / evaluation_queue / evaluation_drafts / results / makeup_slots / + 7 presentation-cycle tables`

GOVERNANCE

`governance_audit / communication_history / communication_templates / deadlines`

Row Level Security is enabled at the Postgres level beneath the application's own checks. The schema evolves through numbered SQL migrations, each carrying a comment explaining the failure it fixes. The schema documents its own history.

DATA / THE KEYSTONE

semester_id carries the whole system

Almost every table carries this column. It is what lets third-year and fourth-year cohorts occupy the same tables concurrently without contaminating each other.

WHAT IT ENABLES

- Two cohorts running different phases simultaneously
- Historical semesters kept intact and queryable
- A global filter that scopes every screen to one batch
- Server-side scoping: the API filters before data leaves the database

MASTER DATA

The **`students`** table is the authoritative roster. Nothing else creates a student. Signup validates against it, and every authenticated request re-checks that the token's email still resolves to a row. Remove someone from master data and their sessions stop working.

DATA / MASTER DATA

One spreadsheet becomes three tables



Every row is stamped with the selected semester_id. This is what keeps cohorts apart.

The file never reaches the backend; only the three upserts do.

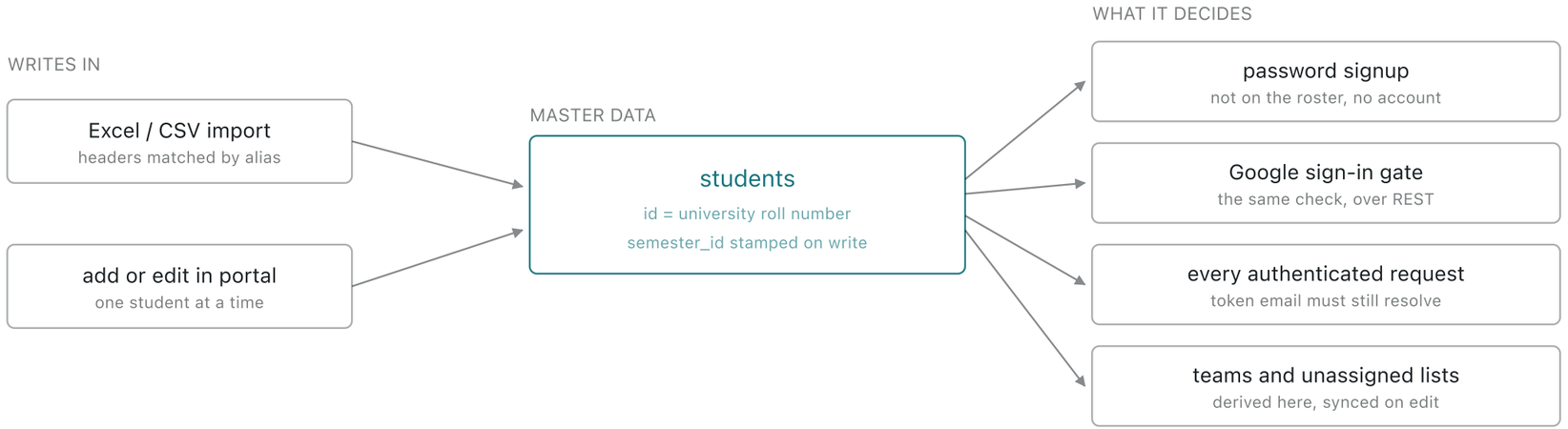
The roster upload fans out on a single decision: does the row carry a team number?

Column headers are resolved through an **alias table**: `E-mail`, `Email ID` and `Mail` all land in one field, because sheets never arrive in a fixed shape.

DATA / THE SINGLE SOURCE OF TRUTH

Master data: one roster the platform trusts

A student comes into existence in exactly one place, and the platform reads that one table as its answer to a single question: is this a real student in this cohort?



Two ways in, one record, and every downstream check pointed back at it.

THE ROLL NUMBER IS THE KEY

`students.id` is the university roll number, so every write is an upsert on that column. Re-importing last week's sheet corrects rows instead of doubling them.

ONE EDIT, THREE TABLES

A name corrected on the master record is rewritten into the `teams_validation` JSONB array and the `unassigned_students` row, so the copies cannot drift.

COHORT-SCOPED ON WRITE

The active `semester_id` is applied by the write layer rather than trusted from the caller, so two batches share one table without seeing each other.

PORTAL 01 / 25 SCREENS

Admin: the control plane

NAVIGATION: 19 NAV ENTRIES / 25 SCREENS

`Dashboard` `Students` `Teams` `Project Allocation` `Mentor Allocation` `Evaluations` `Results` `Governance` `+11 more`

SETUP

- Semester Settings
- Student Upload: Excel roster ingest
- Student List: master data
- Team Validation

ALLOCATION

- Project Allocation + conflict resolution
- Mentor Allocation
- Proposal List & Viewer

ASSESSMENT

- Presentation Cycles
- Schedule / Evaluations
- Results / Absent Students

OVERSIGHT

- Governance: deadlines & data freeze
- Analysis / Repo Analysis / Views

COMMUNICATION

- Segmented mail merge
- Per-Team Mail with merge fields

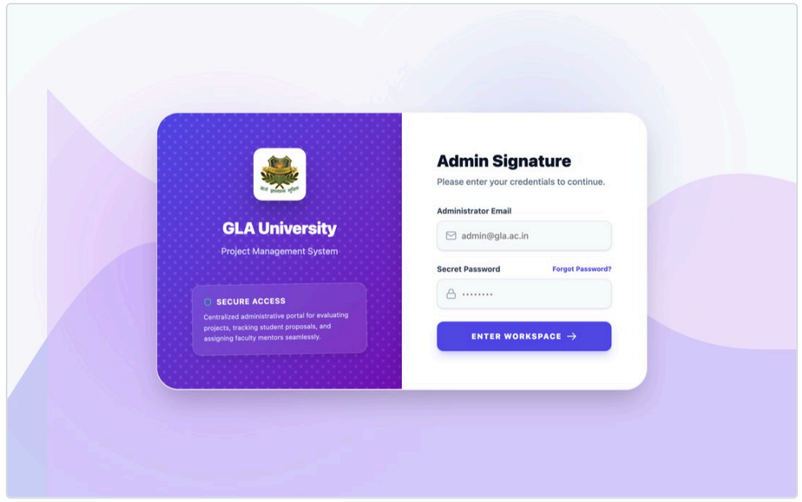
AUDIENCE SEGMENTS

All students / team leaders / unassigned / incomplete teams / teams without mentors / all mentors / one specific team

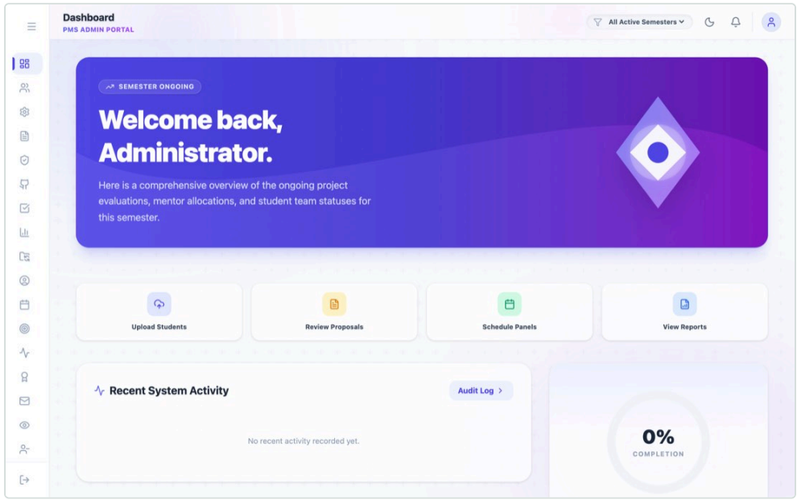
Data freeze is the notable control: one switch locks the entire platform against mutation once grading closes.

PORTAL 01 / CAPTURED FROM THE RUNNING APP

Admin: sign-in and dashboard



Sign-in



Dashboard: quick actions, system activity, completion ring

Captured locally with no database attached. Each panel shows its genuine empty state rather than sample data.

PORTAL 02 / 19 SCREENS

Student: the journey

NAVIGATION: 11 NAV ENTRIES / 19 SCREENS

Dashboard | Team | Project Preferences | Synopsis Submission | Repository Analysis | Evaluation | Results | +4 more

PHASE 1 / JOIN

- Sign up: email confirmation required
- Team Proposal
- Preferences: rank statements by domain and Hardware/Software

PHASE 2 / BUILD

- Milestones
- GitHub Analysis
- GitHub Report

PHASE 3 / ASSESSED

- Presentations
- Evaluations
- Results / Feedback

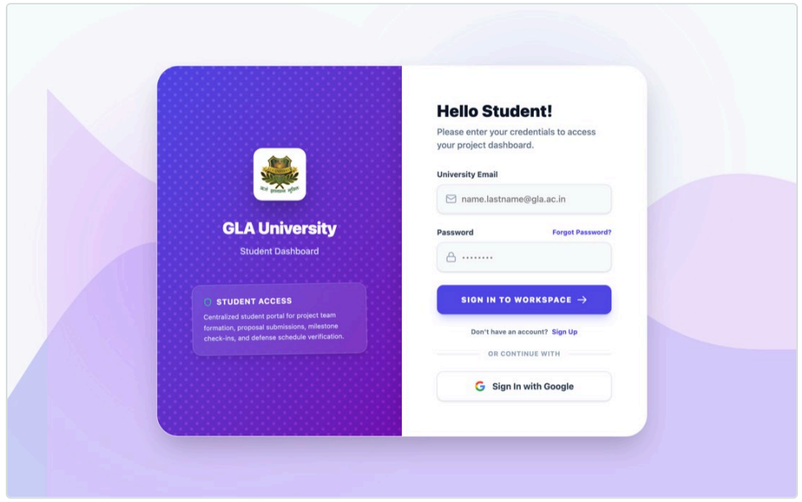
THROUGHOUT

- Extra Activities: certificates for credit
- Mentor / Notifications / Profile

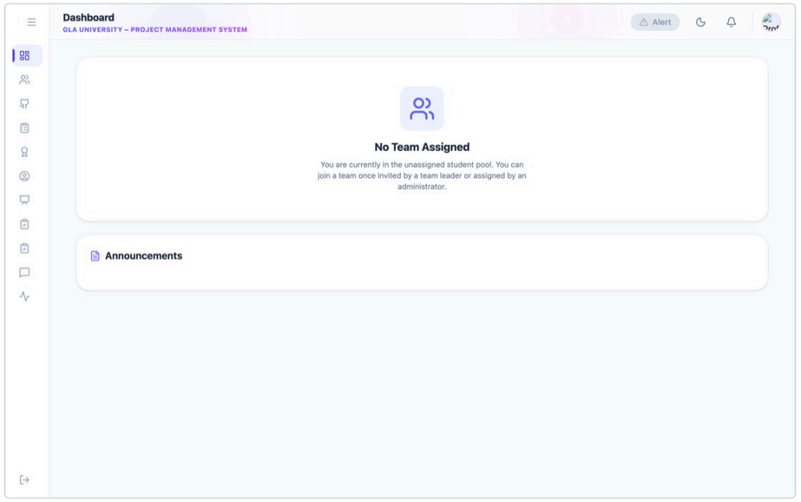
Repository analysis reports commit distribution per contributor. That is how individual contribution *inside* a team is measured objectively rather than by assertion.

PORTAL 02 / CAPTURED FROM THE RUNNING APP

Student: sign-in and dashboard



Sign-in



Dashboard: the empty state a student sees before joining a team

Captured locally with no database attached. Each panel shows its genuine empty state rather than sample data.

PORTAL 03 / 14 SCREENS

Mentor: supervision at scale

NAVIGATION: 7 NAV ENTRIES / 14 SCREENS

`Dashboard` `My Teams` `GitHub Analytics Hub` `Synopsis Submissions` `Presentations` `Evaluations` `History`

THE DAILY SURFACE

- **Assigned Teams** and **Detailed Team View**
- **Milestones Manager**: review submissions
- **Mentor Evaluation Form**: scoring
- **GitHub Analyzer**: repo intelligence across the portfolio

THE DIFFERENTIATORS

Progress Intelligence

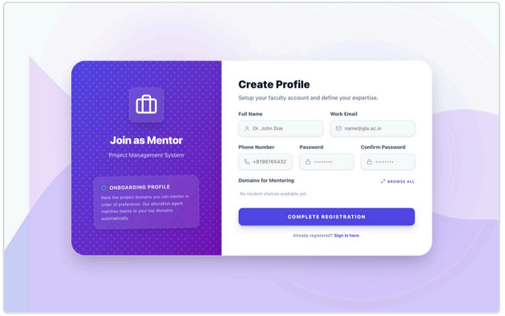
Aggregates a mentor's teams into one comparative view of who is on track.

Alerts

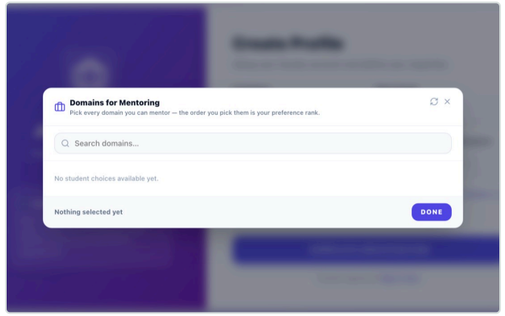
Surfaces teams that have gone quiet: no commits, missed milestones. A mentor supervising eight teams learns which one needs attention without checking all eight.

PORTAL 03 / CAPTURED FROM THE RUNNING APP

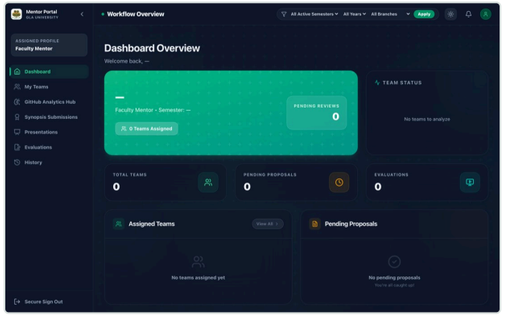
Mentor: sign-up and domain selection



Sign-up: the faculty profile, created by the mentor



Domain selection: pick order is the preference rank



Dashboard: assigned teams, pending proposals, evaluation counts

Captured locally with no database attached. Each panel shows its genuine empty state rather than sample data.

PORTAL 04 / 11 SCREENS

Panel: evaluation under pressure

The narrowest and most disciplined portal, because it is used against the clock with an audience watching.

NAVIGATION: 5 NAV ENTRIES / 11 SCREENS

Dashboard | My Teams | My Slots | Parameters | Missed

STEP 1

Timer starts

A **30-minute** server-side session per team.

STEP 2

Draft autosave

Scores persist continuously; a browser crash loses nothing.

STEP 3

Submit locks

The draft is sealed; further writes are rejected outright.

STEP 4

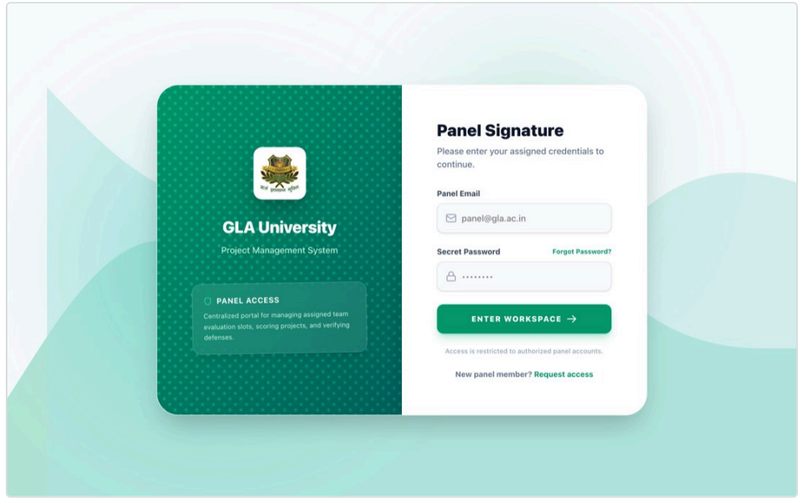
Receipt issued

A receipt ID proves what was submitted, and when.

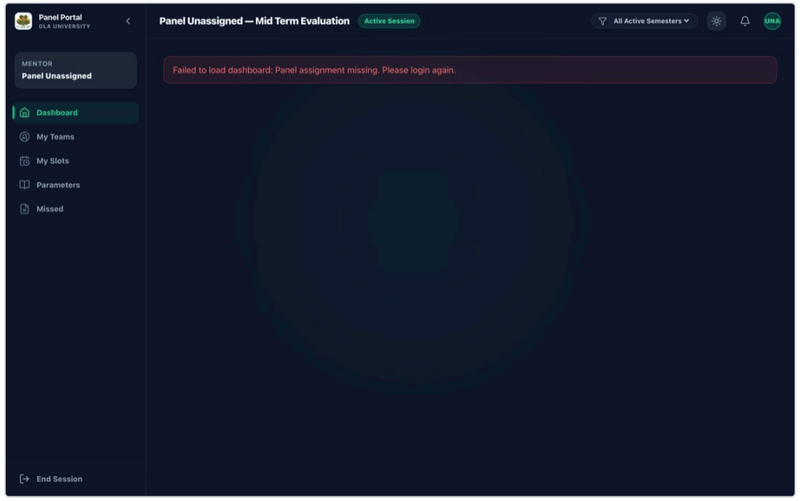
The rubric is **built per team** from the presentation cycle's configuration, not hardcoded. **Missed Submissions** catches evaluations started but never completed.

PORTAL 04 / CAPTURED FROM THE RUNNING APP

Panel: sign-in and dashboard



Sign-in

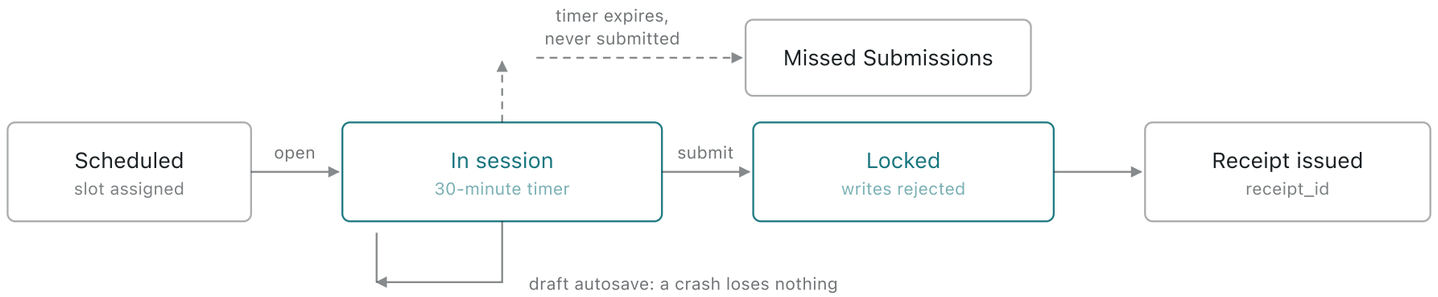


Dashboard: active evaluation session and slot queue

Captured locally with no database attached. Each panel shows its genuine empty state rather than sample data.

ENGINEERING / EVALUATION INTEGRITY

A one-way transition, on the clock



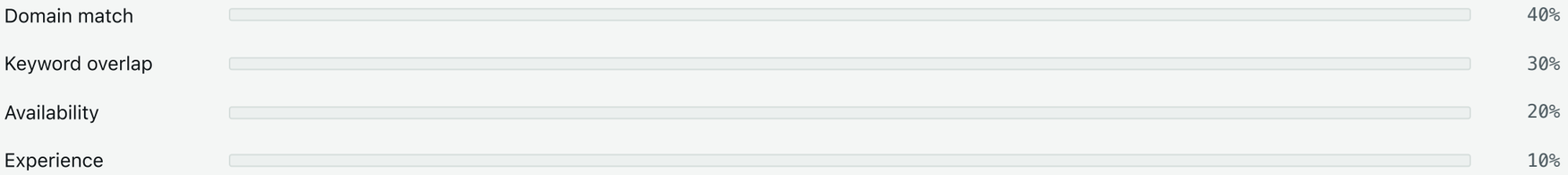
The rubric is assembled per team from the presentation cycle configuration, not hardcoded.

The lock is the point: once submitted, the draft cannot be edited, and the receipt records what was sent.

ENGINEERING / 01

Mentor allocation: deliberately not AI

Matching sixty teams to twenty mentors by hand is a day's work and a bad answer.
This is a deterministic weighted scorer.



DOMAIN TAXONOMY

Related terms are clustered, so semantically-adjacent expertise still scores as a match.

LOAD BALANCING

Availability is current teams against declared maximum, so the algorithm distributes naturally.

EXPLAINABILITY

Every recommendation returns a score breakdown and a human-readable reason. A coordinator overriding it can see what it was thinking.

Why no embeddings? Without a reliable AI key an embedding engine collapses to rule-only *silently*. Better to commit to the rule engine than ship something that degrades invisibly.

ENGINEERING / ALLOCATION, CONTINUED

Why the domain weight needs a taxonomy



A plain string comparison scores these two terms at zero. Clustering is what makes the 40% domain weight meaningful.

How the 40% domain weight is actually computed: through a taxonomy, not string equality.

ENGINEERING / 02

Agents & background schedulers

AGENT MODULES: ADMIN SERVICE

communication	Segmented mail merge across seven audience types
analysis	Aggregate reporting
notification	Portal notification dispatch
deadline	Deadline evaluation and firing

RUNNING SCHEDULERS

presentation reminders	Email and Twilio voice calls ahead of evaluation slots
github sync	Pulls repository activity on a cycle so analysis is never cold
communication	Queued and scheduled sends

ENGINEERING / 03

Security review: 13 findings, all closed

CRITICAL

- **Admin API had no authentication**: every endpoint reachable unauthenticated, and a service-role key meant RLS gave no protection
- Unauthenticated creation and deletion of administrators
- Live production credentials committed to the deploy workflow

HIGH

- Mentor tokens satisfied admin-only routes
- Signup could claim an un-onboarded classmate's account
- Stored XSS through the mail merge
- Reset tokens signed with a leaked key

MEDIUM

- SSRF: a `"github.com"` substring test left the host attacker-controlled
- Any member could remove any teammate
- Logout did not revoke tokens

The admin fix registers the guard **on the application**, not per route, so a future forgotten decorator cannot reopen the hole. None of these were exotic; they are the ordinary gaps that appear when a system grows fast across four services.

CLOSE

What the platform actually changed

A process that ran on spreadsheets and email threads (team formation, project allocation, mentor matching, scheduled evaluation, grading) now gives every one of those steps an audit trail, a deadline, and a defined owner.

4	313	25+	13
PORTALS	ENDPOINTS	TABLES	VULNS CLOSED

The engineering is in service of that. Questions welcome.